Predicting Hospital Readmission in Diabetes Patients

Clinical Complexity, Healthcare Utilization, and Model Performance

Darren Colianni

2026-08-27

Table of contents

1 Introduction **2**
1.1 Data Citation . . . 2

2 Data Cleaning **3**
2.1 Reworking Medical Specialty Predictor . . . 4
2.2 Reworking Diagnoses Predictors . . . 4
2.2.1 Removing Uncommon ones and diag1-3 . . . 5
2.3 Missingness and No Utility Predictors . . . 6
2.4 Extracting Useful Treatments . . . 7
2.5 Final Dataset . . . 8

3 Codebook **9**

4 EDA **12**
4.1 Number of Observations and Missingness . . . 12
4.2 Medical Specialty . . . 14
4.3 Length of Stay . . . 15
4.4 Overall Correlations . . . 17

5 Modeling **18**
5.1 Train/Test Split, Cross-Validation Setup, and Recipe Creation . . . 18
5.2 Model Training . . . 19
5.2.1 1: Elastic Net . . . 20
5.2.2 2: LDA . . . 20
5.2.3 3: Random Forest . . . 21
5.2.4 4: XGBoost . . . 22

5.3 Final Evaluation 23
5.3.1 ROC AUC Summaries 23
5.3.2 Best Model: Random Forest 24
5.3.3 Predictor Importance 25

6 Conclusion **26**
6.1 Potential Extensions 27

1 Introduction

Hospital readmission is a major concern in healthcare because it is associated with increased costs, resource utilization, and poorer patient outcomes. Patients with diabetes often require ongoing management and may experience complications that increase their likelihood of returning to the hospital shortly after discharge.

The goal of this project is to develop machine learning models that predict whether a patient with diabetes will be readmitted to the hospital within 30 days of discharge. This project therefore involves binary classification; readmission, the response variable, is either within 30 days or not. By identifying patients at elevated risk of readmission, healthcare providers may be able to target interventions and improve patient outcomes.

Predictor variables include demographic characteristics (race, gender, age), admission information (admission type, discharge disposition, admission source), laboratory measurements (serum glucose, A1C), diabetes medication indicators, healthcare utilization measures (number of inpatient, outpatient, and emergency visits), and comorbidity indicators derived from diagnosis codes. Several classification models are fit and compared to determine which approach best predicts patient readmission.

The data source is the *Diabetes 130-US Hospitals for Years 1999–2008* dataset from the UCI Machine Learning Repository. It contains over 100,000 hospital encounters from 130 U.S. hospitals and integrated delivery networks between 1999 and 2008. Because the same patient may appear multiple times, only the most recent encounter per patient is retained, yielding 71,518 observations. The outcome is severely imbalanced: approximately 4.5% of encounters result in a readmission within 30 days.

1.1 Data Citation

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). *Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records. BioMed Research International*, 2014, 781670.

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Diabetes 130-US Hospitals for Years 1999–2008. UCI Machine Learning Repository. Available at: https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008.

```r
# Loading in basic packages
library(ggplot2)
library(dplyr)
library(tidymodels)
tidymodels_prefer()

# Loading in saved results from data cleaning
load("saves/data.rda")
```

```r
# Reading in the data
og_data <- read.csv("data/diabetic_data.csv")
og_data <- og_data %>%
  group_by(patient_nbr) %>%
  slice_max(encounter_id) %>%
  ungroup()
```

2 Data Cleaning

There is a good deal of no/low utility predictors, or those which encode too many values and need to be transformed for better comprehensibility and modeling.

But first, we will begin by making all missing values "NA," and redefine the readmitted variable as readmitted within 30 days (given the value 1) or not (given the value 0).

```r
cleaned_data <- og_data

# Missing values were largely coded as "?". Gender has 3 "Unknown" observations...
   ## ... which we will replace with "NA".
cleaned_data[cleaned_data == "?"] <- NA
cleaned_data <- cleaned_data %>% mutate(gender = na_if(gender, "Unknown/Invalid"))

# Making readmitted a binary factor
cleaned_data <- cleaned_data %>%
  mutate(readmitted = as.factor(ifelse(readmitted == "<30", 1, 0)))
```

2.1 Reworking Medical Specialty Predictor

The medical specialty predictor has too many categories (84), as one can see from the original datasource codebook. We'll keep only some of the top specialties and group the rest into "Other" to prevent overfitting.

```
# Keep categories with at least 1% of the total observations.
library(forcats)
cleaned_data1 <- cleaned_data %>%
  mutate(
    medical_specialty = fct_lump_prop(
      medical_specialty,
      prop = 0.01,
      other_level = "Other"
    )
  )
```

```
table(cleaned_data1$medical_specialty, useNA = "always")
```

```
                 Cardiology           Emergency/Trauma
                       4143                       4463
    Family/GeneralPractice           InternalMedicine
                       5021                      11069
                  Nephrology                Orthopedics
                        887                       1063
Orthopedics-Reconstructive                 Radiologist
                       1044                        819
            Surgery-General                      Other
                       2235                       6249
                       <NA>
                      34525
```

2.2 Reworking Diagnoses Predictors

The diagnoses predictors (diag_1, diag_2, diag_3) have many distinct values (848) and are coded in such a way that they are many different codes. Luckily, AHRQ manually grouped these codes into meaningful buckets, and there's a library for this. We will grab these new codes instead and remove the diag_1…diag_3, which are encoded in these buckets.

```r
library(comorbidity)

ccs1 <- comorbidity(
  x = cleaned_data1,
  id = "patient_nbr",
  code = "diag_1",
  map = "elixhauser_icd9_quan",
  assign0 = FALSE
)

ccs2 <- comorbidity(
  x = cleaned_data1,
  id = "patient_nbr",
  code = "diag_2",
  map = "elixhauser_icd9_quan",
  assign0 = FALSE
)

ccs3 <- comorbidity(
  x = cleaned_data1,
  id = "patient_nbr",
  code = "diag_3",
  map = "elixhauser_icd9_quan",
  assign0 = FALSE
)

ccs_all <- bind_rows(ccs1, ccs2, ccs3) %>%
  group_by(patient_nbr) %>%
  summarise(across(where(is.numeric), max, na.rm = TRUE), .groups = "drop")

cleaned_data2 <- cleaned_data1 %>%
  left_join(ccs_all, by = "patient_nbr")
```

2.2.1 Removing Uncommon ones and diag1-3

We will remove the especially uncommon ones since they won't provide much predictive utility to us.

```r
diag_cols <- c("chf","carit","valv","pcd","pvd","hypunc","hypc",
               "para","ond","cpd","diabunc","diabc","hypothy","rf",
               "ld","pud","aids","lymph","metacanc","solidtum",
```

```
                "rheumd","coag","obes","wloss","fed","blane","dane",
                "alcohol","drug","psycho","depre")

keep_diag <- cleaned_data2 %>%
  summarise(across(all_of(diag_cols), ~ mean(. == 1, na.rm = TRUE))) %>%
  pivot_longer(everything(), names_to = "diag", values_to = "prev") %>%
  filter(prev >= 0.01) %>%
  pull(diag)

cleaned_data3 <- cleaned_data2 %>%
  select(-c(diag_1, diag_2, diag_3),
         -all_of(setdiff(diag_cols, keep_diag)))
```

2.3 Missingness and No Utility Predictors

We will now also remove predictors with extreme missingness or no utility in predicting readmission.

```
cleaned_data3 %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(
    everything(),
    names_to = "predictor",
    values_to = "na_count"
  ) %>%
  arrange(desc(na_count))
```

```
# A tibble: 60 x 2
   predictor                na_count
   <chr>                       <int>
 1 weight                      68671
 2 medical_specialty           34525
 3 payer_code                  30085
 4 race                         1878
 5 gender                          3
 6 encounter_id                    0
 7 patient_nbr                     0
 8 age                             0
 9 admission_type_id               0
10 discharge_disposition_id        0
# i 50 more rows
```

Around half of medical specialties are missing, but we will keep this, as it may still provide some predictive value. We will remove weight, however, which is almost entirely missing, as well as payer_code, patient_nbr, and encounter_id, which have no relation to predicting readmission.

```
cleaned_data4 <- cleaned_data3 %>%
  select(-c(weight, payer_code, patient_nbr, encounter_id))
```

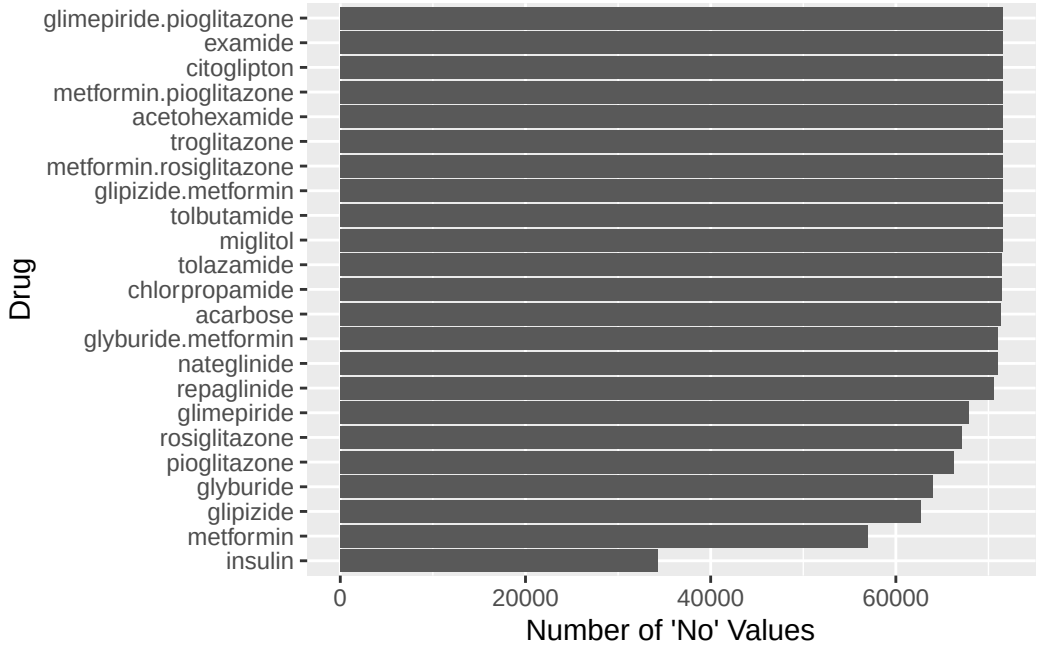
2.4 Extracting Useful Treatments

We start with many diabetes drugs as possible predictors. We will cut this down by examining how one-sided they are. If almost no patients took them, they would be poor predictors.

```
treatments <- c("metformin", "repaglinide", "nateglinide", "chlorpropamide",
                "glimepiride", "acetohexamide", "glipizide", "glyburide",
                "tolbutamide", "pioglitazone", "rosiglitazone", "acarbose",
                "miglitol", "troglitazone", "tolazamide", "examide",
                "citoglipton", "insulin", "glyburide.metformin",
                "glipizide.metformin", "glimepiride.pioglitazone",
                "metformin.rosiglitazone", "metformin.pioglitazone")

no_counts <- cleaned_data4 %>%
  select(all_of(treatments)) %>%
  summarise(across(everything(), ~sum(. == "No"))) %>%
  pivot_longer(everything(), names_to = "drug", values_to = "no_count") %>%
  arrange(desc(no_count))

no_counts %>%
  ggplot(aes(x = reorder(drug, no_count), y = no_count)) +
  geom_col() +
  coord_flip() +
  labs(x = "Drug", y = "Number of 'No' Values")
```



We remove the drugs which are almost all "No" since they provide almost no predictive utility. This removal is a similar idea to the diagnoses, but the caret library has a builtin function which facilitates it.

Additionally, since there are likely other near-zero variance predictors, we will remove them along with these known ones all at once.

```
library(caret)
nzv_cols <- nearZeroVar(cleaned_data4)
nzv_cols <- nzv_cols[names(cleaned_data4)[nzv_cols] != "readmitted"]
diab <- cleaned_data4[, -nzv_cols]

save(og_data, cleaned_data, cleaned_data1, cleaned_data2, cleaned_data3,
     cleaned_data4, diab, file="saves/data.rda")
```

2.5 Final Dataset

Here are the ultimate variables we are working with:

```
names(diab)
```

```
 [1] "race"                      "gender"
 [3] "age"                       "admission_type_id"
```

```
 [5] "discharge_disposition_id" "admission_source_id"
 [7] "time_in_hospital"         "medical_specialty"
 [9] "num_lab_procedures"       "num_procedures"
[11] "num_medications"          "number_outpatient"
[13] "number_emergency"         "number_inpatient"
[15] "number_diagnoses"         "A1Cresult"
[17] "metformin"                "glipizide"
[19] "glyburide"                "pioglitazone"
[21] "rosiglitazone"            "insulin"
[23] "change"                   "diabetesMed"
[25] "readmitted"               "chf"
[27] "hypunc"                   "hypc"
[29] "cpd"                      "diabunc"
[31] "diabc"                    "fed"
```

We will be going into each of these resulting predictors (alongside readmitted) in the codebook below.

3 Codebook

Variable	Description	Values
`race`	Patient race	Caucasian, AfricanAmerican, Hispanic, Asian, Other, NA
`gender`	Patient gender	Male, Female, NA
`age`	Patient age group	[0-10), [10-20), [20-30), [30-40), [40-50), [50-60), [60-70), [70-80), [80-90), [90-100)
`admission_type_id`	Type of hospital admission	1=Emergency, 2=Urgent, 3=Elective, 4=Newborn, 5=Not Available, 6=Not Available, 7=Trauma Center, 8=Not Mapped

Variable	Description	Values
`discharge_disposition_id`	Patient disposition after discharge	Integer identifier; selected values include 1=Discharged to home, 2=Discharged to another short-term hospital, 3=Skilled nursing facility, 4=Intermediate care facility, 5=Another type of inpatient care institution, 6=Home with home health service, 7=Left AMA, 8=Home with IV provider, 9=Admitted as inpatient, 11=Expired, 13=Hospice/home, 14=Hospice/medical facility, 18=Not Mapped, 19–21=Expired variants, 25=Not Mapped, 28=Not Mapped
`admission_source_id`	Source of the hospital admission	Integer identifier; selected values include 1=Physician Referral, 2=Clinic Referral, 3=HMO Referral, 4=Transfer from hospital, 5=Transfer from SNF, 6=Transfer from another health care facility, 7=Emergency Room, 8=Court/Law Enforcement, 9=Not Available, 10=Transfer from critical access hospital, 11=Normal delivery, 13=Sick baby, 14=Extramural birth, 17=NULL, 20=Not Mapped, 22=Transfer from outpatient surgery, 25=Not Mapped
`time_in_hospital`	Length of hospital stay	Integer number of days (1–14)

Variable	Description	Values
medical_specialty	Specialty of the admitting physician	InternalMedicine, Cardiology, Emergency/Trauma, Family/GeneralPractice, Nephrology, Orthopedics, Orthopedics-Reconstructive, Surgery-General, Radiologist, Other, NA
num_lab_procedures	Number of laboratory procedures performed	Integer
num_procedures	Number of non-laboratory procedures performed	Integer
num_medications	Number of medications prescribed	Integer
number_outpatient	Number of outpatient visits in the prior year	Integer
number_emergency	Number of emergency visits in the prior year	Integer
number_inpatient	Number of inpatient visits in the prior year	Integer
number_diagnoses	Number of diagnoses recorded	Integer
max_glu_serum	Maximum serum glucose test result	None=not tested, Norm=normal, >200, >300
A1Cresult	Hemoglobin A1C test result	None=not tested, Norm=normal (7%), >7, >8
metformin	Metformin prescription status	No, Steady, Up, Down
glipizide	Glipizide prescription status	No, Steady, Up, Down
glyburide	Glyburide prescription status	No, Steady, Up, Down
pioglitazone	Pioglitazone prescription status	No, Steady, Up, Down
rosiglitazone	Rosiglitazone prescription status	No, Steady, Up, Down
insulin	Insulin prescription status	No, Steady, Up, Down
change	Whether diabetes medications were changed	Ch=changed, No=no change
diabetesMed	Whether any diabetes medication was prescribed	Yes, No
chf	Congestive heart failure (Elixhauser comorbidity)	0=absent, 1=present

Variable	Description	Values
`hypunc`	Uncomplicated hypertension (Elixhauser comorbidity)	0=absent, 1=present
`hypc`	Complicated hypertension (Elixhauser comorbidity)	0=absent, 1=present
`cpd`	Chronic pulmonary disease (Elixhauser comorbidity)	0=absent, 1=present
`diabunc`	Uncomplicated diabetes (Elixhauser comorbidity)	0=absent, 1=present
`diabc`	Complicated diabetes (Elixhauser comorbidity)	0=absent, 1=present
`fed`	Fluid and electrolyte disorders (Elixhauser comorbidity)	0=absent, 1=present
`readmitted`	Whether the patient was readmitted within 30 days	0=No, 1=Yes

4 EDA

4.1 Number of Observations and Missingness

Number of observations:

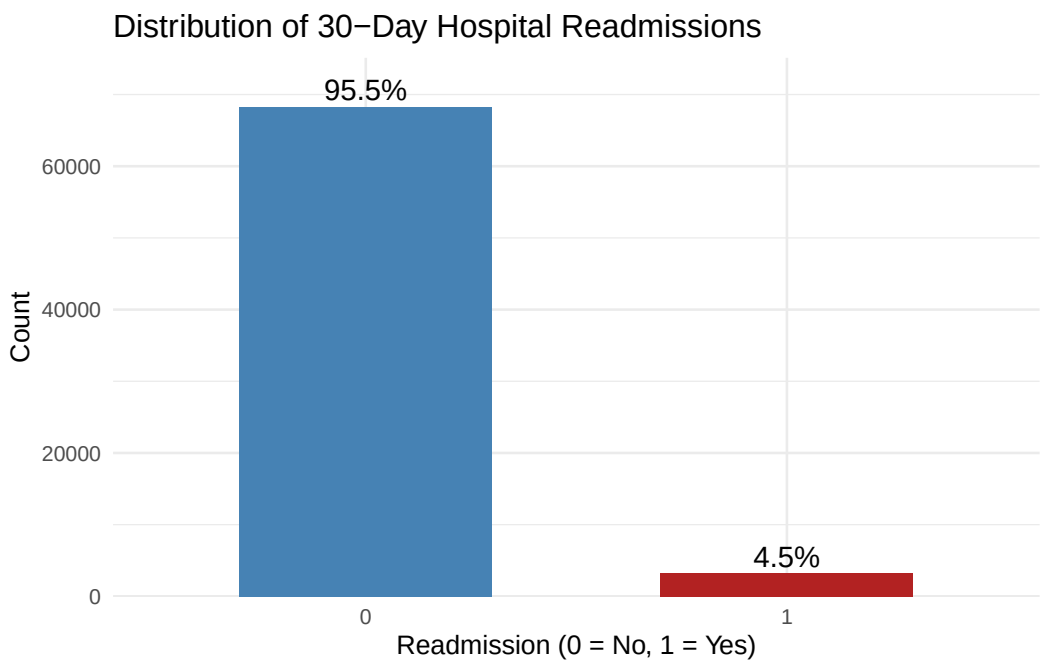
```
nrow(diab)
```

```
[1] 71518
```

Readmission distribution:

```
diab %>%
  ggplot(aes(x = factor(readmitted), fill = factor(readmitted))) +
  geom_bar(width = 0.6) +
  geom_text(
    stat = "count",
    aes(label = scales::percent(after_stat(count) / sum(after_stat(count)),
                                accuracy = 0.1)),
    vjust = -0.3,
    size = 4
  ) +
```

```
  scale_fill_manual(values = c("0" = "steelblue", "1" = "firebrick")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.1))) +
  labs(
    title = "Distribution of 30-Day Hospital Readmissions",
    x = "Readmission (0 = No, 1 = Yes)",
    y = "Count",
    fill = "Readmitted"
  ) +
  theme_minimal(base_size = 10) +
  theme(legend.position = "none")
```



Missingness:

```
diab %>% is.na() %>% colSums() %>% .[. > 0]
```

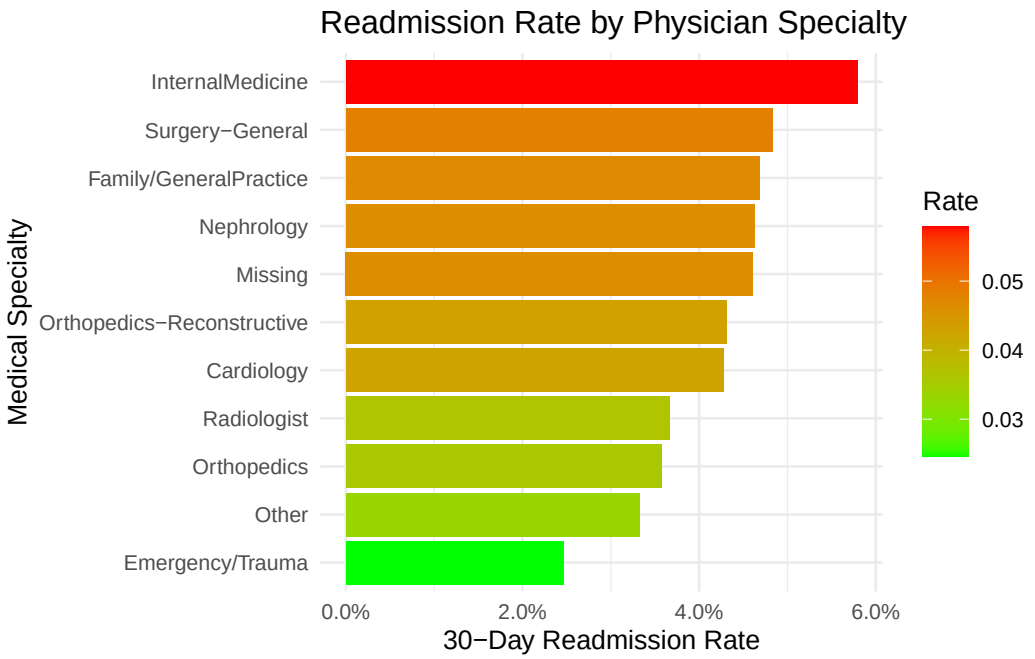
```
            race           gender medical_specialty
            1878                3            34525
```

Because the variables with missingness are categorical, their values will be given an explicit "Unknown" category using step_unknown() during preprocessing (recipe formation). As categorical variables have no natural notion of a central value, imputation would have required the hefty assumption on which category each missing value should have been assigned.

4.2 Medical Specialty

```
library(scales)

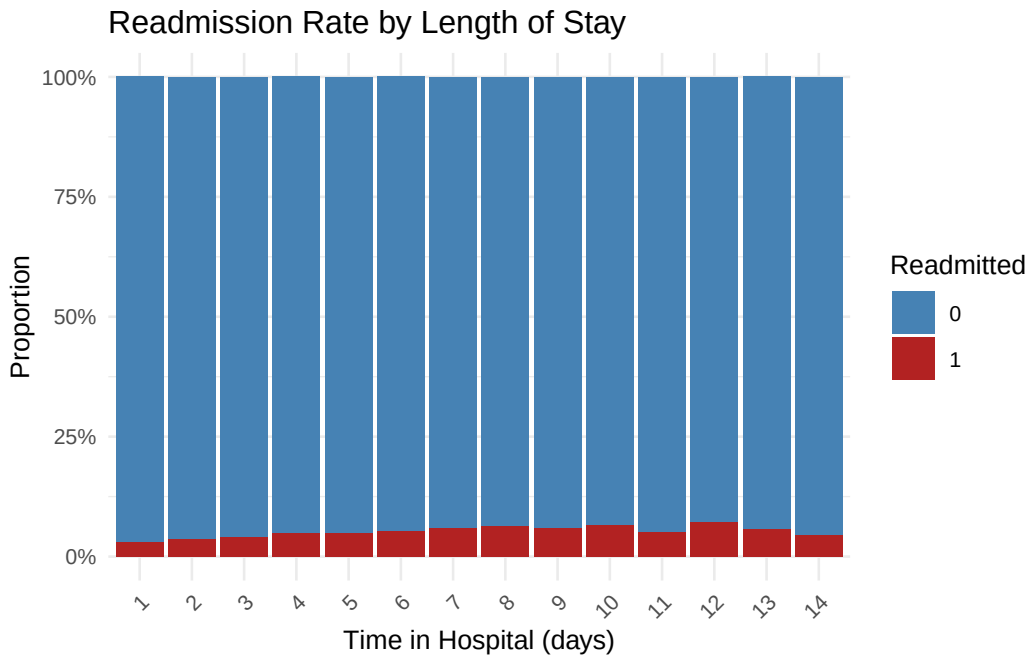
diab %>%
  mutate(medical_specialty = forcats::fct_na_value_to_level(medical_specialty,
                                                             level = "Missing")) %>%
  group_by(medical_specialty) %>%
  summarise(readmit_rate = mean(readmitted == 1, na.rm = TRUE)) %>%
  ungroup() %>%
  mutate(medical_specialty = reorder(medical_specialty, readmit_rate)) %>%
  ggplot(aes(x = medical_specialty,
             y = readmit_rate,
             fill = readmit_rate)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format(accuracy = 0.1)) +
  scale_fill_gradient(low = "green", high = "red") +
  labs(
    title = "Readmission Rate by Physician Specialty",
    x = "Medical Specialty",
    y = "30-Day Readmission Rate",
    fill = "Rate"
  ) +
  theme_minimal(base_size = 10) +
  theme(legend.position = "right")
```



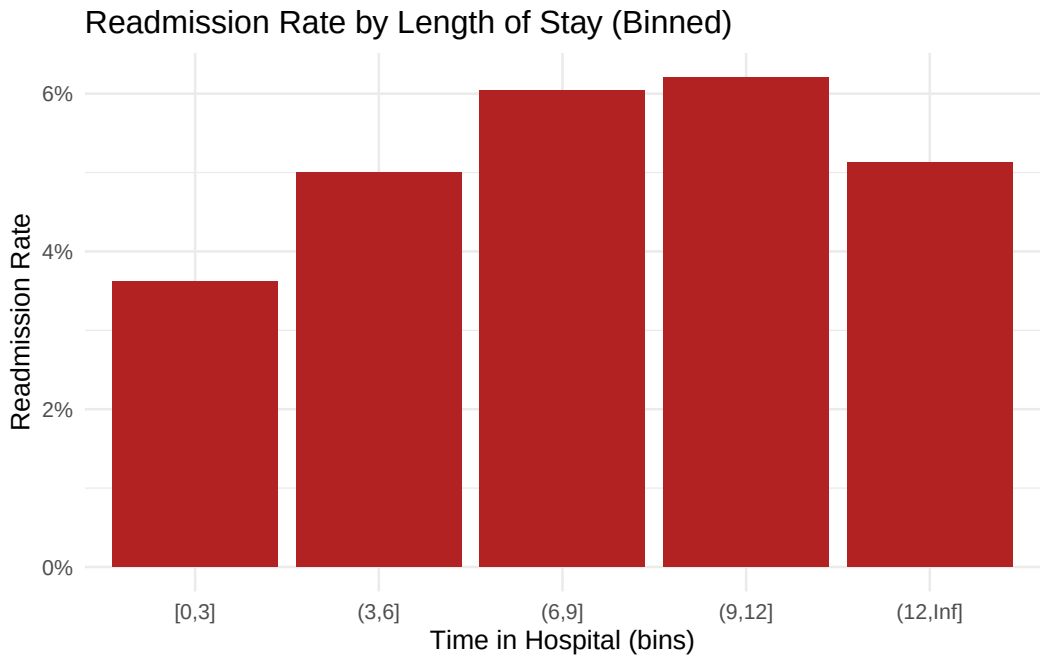
As expected, medical specialty is somewhat informative. In the above bar chart with the readmission proportions, different specialty mean proportions span from around 2.5% to 6%, which is relatively quite informative given the overall rarity of readmissions.

4.3 Length of Stay

```r
diab %>%
  ggplot(aes(x = factor(time_in_hospital), fill = factor(readmitted))) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(values = c("0" = "steelblue", "1" = "firebrick")) +
  labs(
    title = "Readmission Rate by Length of Stay",
    x = "Time in Hospital (days)",
    y = "Proportion",
    fill = "Readmitted"
  ) +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
diab %>%
  mutate(time_bin = cut(time_in_hospital,
                        breaks = c(0, 3, 6, 9, 12, Inf),
                        include.lowest = TRUE)) %>%
  group_by(time_bin) %>%
  summarise(readmit_rate = mean(readmitted == 1, na.rm = TRUE)) %>%
  ggplot(aes(x = time_bin, y = readmit_rate)) +
  geom_col(fill = "firebrick") +
  scale_y_continuous(labels = percent) +
  labs(
    title = "Readmission Rate by Length of Stay (Binned)",
    x = "Time in Hospital (bins)",
    y = "Readmission Rate"
  ) +
  theme_minimal(base_size = 10)
```



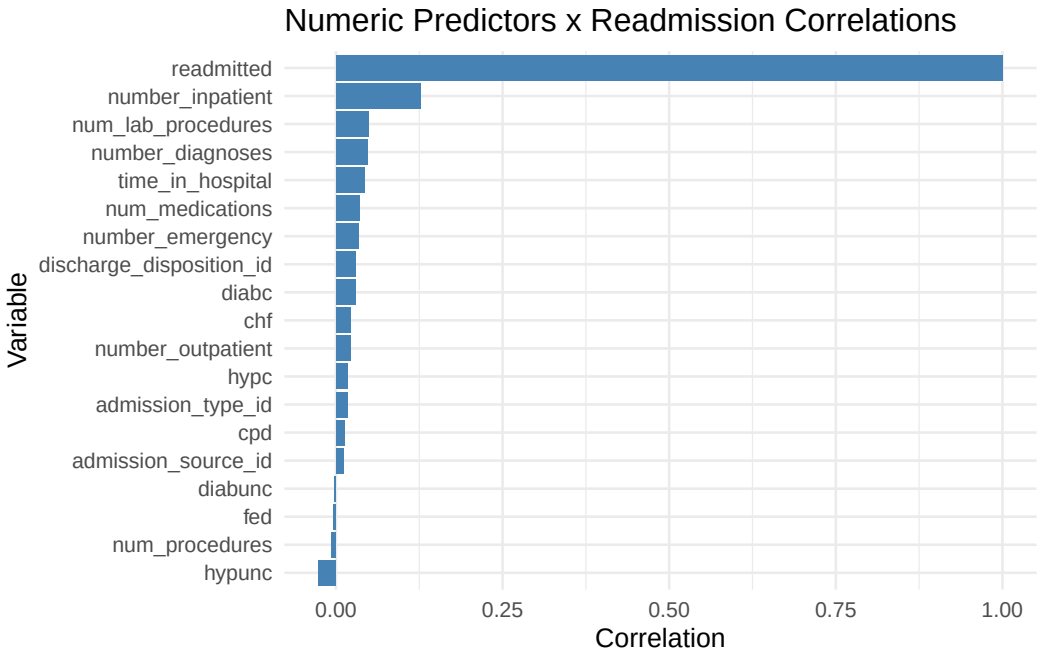
From approximately 1 to 8 days, readmission rates increase with length of stay. Beyond this range, the trend appears to level off, with no consistent further increase and possible mild decreases at longer stays; this potential trend may reflect improved stabilization and discharge timing for more severe cases.

4.4 Overall Correlations

```r
num_cor <- diab %>%
  mutate(readmitted = as.numeric(readmitted)) %>%
  select(where(is.numeric)) %>%
  summarise(across(everything(), ~ cor(.x, readmitted,
                                       use = "pairwise.complete.obs"))) %>%
  tidyr::pivot_longer(cols = everything(),
                      names_to = "variable",
                      values_to = "correlation") %>%
  arrange(desc(abs(correlation)))

ggplot(num_cor, aes(x = reorder(variable, correlation), y = correlation)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Numeric Predictors x Readmission Correlations",
```

```
    x = "Variable",
    y = "Correlation"
  ) +
  theme_minimal(base_size = 10)
```



The most correlated predictors indicate disease severity most directly, like the number of inpatient admissions in the past year or other hospital-stay related factors. Overall, these low individual correlations indicates that any direct linear relationship to readmissions is weak, and a tree-based model will therefore likely perform best.

5 Modeling

5.1 Train/Test Split, Cross-Validation Setup, and Recipe Creation

The cleaned dataset is partitioned into a training set (75%) and a testing set (25%), stratified by the outcome to preserve the class distribution in both sets. The training set is used exclusively for model fitting and tuning; the testing set is held out and used only once, for final evaluation of the best model.

To tune hyperparameters without touching the test set, 5-fold cross-validation is applied to the training data. The training set is split into 5 roughly equal folds, also stratified by outcome. Each model is trained on 4 folds and evaluated on the remaining fold, rotating until every fold

has served as the validation set. Performance is averaged across all 5 folds to produce a stable estimate. This procedure prevents overfitting to any particular subset of the training data and gives a reliable estimate of how each model will generalize.

A preprocessing recipe is applied consistently within each fold to prevent data leakage: missing categorical values are assigned an explicit "Unknown" level, nominal predictors are one-hot encoded, three interaction terms are created between key utilization variables, and all numeric predictors are normalized. Downsampling of the majority class is also applied within each fold at a 10:1 majority-to-minority ratio to reduce training time.

```
library(themis)

set.seed(0)

split <- initial_split(diab, prop = 0.75, strata = readmitted)
train <- training(split)
test  <- testing(split)

folds <- vfold_cv(train, v = 5, strata = readmitted)

rec <- recipe(readmitted ~ ., data = train) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~
    number_inpatient:time_in_hospital +
    num_medications:number_diagnoses +
    number_inpatient:num_medications
  ) %>%
  step_downsample(readmitted, under_ratio = 10) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())

cv_metrics <- metric_set(roc_auc)
```

5.2 Model Training

Four classification models are fit and compared. Elastic net logistic regression applies L1 and L2 regularization simultaneously, with the penalty and mixture parameters tuned over a regular grid of 25 combinations. Linear discriminant analysis (LDA) assumes class-conditional Gaussian distributions with equal covariance; because the full feature set introduces collinearity, a PCA-based recipe reducing to however many components to explain 95% of the variance in the feature set. Random forest fits 300 decision trees and averages their predictions; the

number of predictors considered at each split (mtry) and the minimum node size (min_n) are tuned over a 4x4 regular grid. XGBoost fits trees sequentially, each correcting residuals of the last; tree depth, learning rate, and mtry are tuned over 20 space-filling grid points. All models are evaluated using ROC AUC averaged across the 5 cross-validation folds.

5.2.1 1: Elastic Net

```
en_spec <- logistic_reg(penalty = tune(), mixture = tune()) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

en_wf <- workflow() %>%
  add_recipe(rec) %>%
  add_model(en_spec)

en_grid <- grid_regular(penalty(), mixture(), levels = 5)
```

```
en_res <- tune_grid(
  en_wf,
  resamples = folds,
  grid = en_grid,
  metrics = cv_metrics
)

en_best <- select_best(en_res, metric = "roc_auc")

save(
  en_res, en_best,
  file = "saves/model_results1.rda"
)
```

5.2.2 2: LDA

```
# Some collinearity in the original recipe. LDA-specific recipe using PCA
lda_rec <- recipe(readmitted ~ ., data = train) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
```

```
  step_pca(all_numeric_predictors(), threshold =  .95)  # explain 95% of var

library(discrim)

lda_spec <- discrim_linear() %>%
  set_engine("MASS") %>%
  set_mode("classification")

lda_wf <- workflow() %>%
  add_recipe(lda_rec) %>%
  add_model(lda_spec)
```

```
lda_res <- fit_resamples(
  lda_wf,
  resamples = folds,
  metrics = cv_metrics
)

save(lda_res, file = "saves/model_results2.rda")
```

5.2.3 3: Random Forest

```
rf_spec <- rand_forest(
  mtry  = tune(),
  trees = 300,
  min_n = tune()
) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_wf <- workflow() %>%
  add_recipe(rec) %>%
  add_model(rf_spec)

rf_grid <- grid_regular(
  mtry(range = c(3, 20)),
  min_n(range = c(2, 10)),
  levels = 4
)
```

```r
library(doParallel)

cl <- makePSOCKcluster(parallel::detectCores() - 1)
on.exit(stopCluster(cl), add = TRUE)
registerDoParallel(cl)

rf_res <- tune_grid(
  rf_wf,
  resamples = folds,
  grid = rf_grid,
  metrics = cv_metrics
)

rf_best <- select_best(rf_res, metric = "roc_auc")

save(
  rf_res,
  rf_best,
  file = "saves/model_results3.rda"
)
```

5.2.4 4: XGBoost

```r
library(xgboost)

xgb_spec <- boost_tree(
  trees      = 300,
  tree_depth = tune(),
  learn_rate = tune(),
  mtry       = tune()
) %>%
  set_engine("xgboost", nthread = 1) %>%
  set_mode("classification")

xgb_wf <- workflow() %>%
  add_recipe(rec) %>%
  add_model(xgb_spec)

xgb_grid <- grid_space_filling(
  tree_depth(range = c(2, 8)),
```

```
  learn_rate(range = c(0.01, 0.3)),
  mtry(range = c(3, 20)),
  size = 20
)
```

```
xgb_res <- tune_grid(
  xgb_wf,
  resamples = folds,
  grid = xgb_grid,
  metrics = cv_metrics
)

xgb_best <- select_best(xgb_res, metric = "roc_auc")

save(
  xgb_res,
  xgb_best,
  file = "saves/model_results4.rda"
)
```

```
save(
  en_res, lda_res, rf_res, xgb_res,
  en_best, rf_best, xgb_best,
  file = "saves/model_results.rda"
)
```

5.3 Final Evaluation

```
load("saves/model_results.rda")
```

5.3.1 ROC AUC Summaries

Below are the metrics for each of our fitted models across different hyperparameters:

```
bind_rows(
  collect_metrics(en_res)  %>% mutate(model = "Elastic Net"),
  collect_metrics(lda_res) %>% mutate(model = "LDA"),
  collect_metrics(rf_res)  %>% mutate(model = "Random Forest"),
  collect_metrics(xgb_res) %>% mutate(model = "XGBoost")
```

```
) %>%
  filter(.metric == "roc_auc") %>%
  group_by(model) %>%
  slice_max(mean, n = 1) %>%
  arrange(desc(mean))
```

```
# A tibble: 4 x 13
# Groups:   model [4]
  penalty mixture .metric .estimator  mean     n std_err .config     model  mtry
    <dbl>   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>       <chr> <int>
1      NA      NA roc_auc binary     0.671     5 0.00680 pre0_mod04~ Rand~     3
2       1       0 roc_auc binary     0.660     5 0.00649 pre0_mod21~ Elas~    NA
3      NA      NA roc_auc binary     0.628     5 0.00493 pre0_mod0_~ LDA      NA
4      NA      NA roc_auc binary     0.628     5 0.00722 pre0_mod12~ XGBo~    12
# i 3 more variables: min_n <int>, tree_depth <int>, learn_rate <dbl>
```

5.3.2 Best Model: Random Forest

The one which performed best was the random forest model, which we will now compare with the testing set ROC AUC performance.

```
final_rf_fit <- rf_wf %>%
  finalize_workflow(rf_best) %>%
  fit(data = train)

rf_preds <- predict(final_rf_fit, test, type = "prob") %>%
  bind_cols(test)

roc_auc(rf_preds, truth = readmitted, .pred_1, event_level = "second")
```

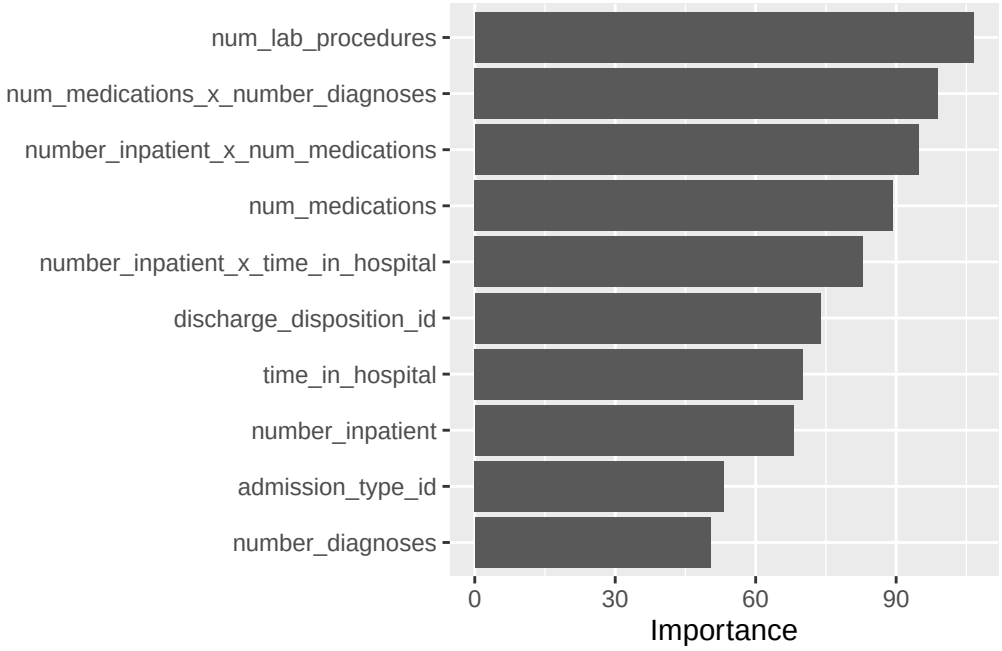
```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 roc_auc binary         0.695
```

The cross-validated ROC AUC summaries above show each model's best cross-validated ROC AUC. Random forest achieved the highest, followed by XGBoost, elastic net, and LDA. The testing set ROC AUC of 0.695 is consistent with the training ROC AUC of 0.67, indicating the model generalizes well with no meaningful overfitting. Performance in the 0.65–0.70 range is consistent with published results on this dataset, where the inherently low signal in 30-day readmission data places a practical ceiling on predictive accuracy regardless of model choice.

5.3.3 Predictor Importance

```
library(vip)

vip(final_rf_fit)
```




```
vip::vi(final_rf_fit) %>% slice_head(n = 20)
```

```
# A tibble: 20 x 2
   Variable                              Importance
   <chr>                                      <dbl>
 1 num_lab_procedures                         106.
 2 num_medications_x_number_diagnoses          98.9
 3 number_inpatient_x_num_medications          94.8
 4 num_medications                             89.4
 5 number_inpatient_x_time_in_hospital         83.0
 6 discharge_disposition_id                    73.9
 7 time_in_hospital                            70.1
 8 number_inpatient                            68.1
 9 admission_type_id                           53.2
10 number_diagnoses                            50.3
11 num_procedures                              47.5
```

```
12 admission_source_id                    39.5
13 number_outpatient                      38.5
14 number_emergency                       31.7
15 gender_Male                            19.3
16 medical_specialty_InternalMedicine     17.9
17 diabc                                  17.9
18 medical_specialty_unknown              17.8
19 chf                                    17.4
20 age_X.70.80.                           17.2
```

Healthcare utilization and illness severity matter more than demographics. Some of the strongest predictors also include their interaction terms like num_medications: number_diagnoses and number_inpatient: time_in_hospital, indicating that readmission risk increases when patients have both high medication burden and multiple conditions, or repeated inpatient stays combined with longer hospitalizations.

Also of high importance are clinical intensity variables like num_lab_procedures, num_medications, number_inpatient, and time_in_hospital; ie, sicker people will be readmitted more.

Discharge disposition and prior utilization measures like emergency and outpatient visits also add signal, suggesting post-discharge care path and historical healthcare usage are important for risk.

6 Conclusion

The feature rankings indicate that cumulative disease burden and repeated healthcare engagement matter more than static patient characteristics. The strongest predictors are forms of healthcare utilization and clinical complexity (e.g., No. of inpatient visits, length of stay, number of diagnoses, medication burden, and interactions therein). Contrastingly, demographic features contributed relatively little.

Across the models tested, random forest achieved the best overall ROC-AUC performance, with the highest mean score among all methods evaluated. Variability between folds was also reasonable, at approximately 0.00680. Meanwhile, linear models such as elastic net and LDA performed more modestly, reflecting their limited capacity to capture nonlinear relationships and higher-order interactions present in the clinical features. Also, though gradient boosted trees were included as a more flexible ensemble method, they did not outperform the random forest in this implementation, perhaps due to hyperparameter tuning sensitivity or merely the structure of the feature space/interactions favoring averaging as opposed to sequential correction.

6.1 Potential Extensions

A more extensive modeling search may include additional boosting methods. Alternatives include LightGBM or CatBoost; these may handle complex interactions and imbalance in this dataset better. Barring that possibility, the only other improvements are structurally difficult, such as having hospitals gather more relevant data about the patients to result in a richer set of predictors.